

UPPER BOUNDS ON ARITHMETIC TURING DEGREES

MILES KRETSCHMER

1. ARITHMETIC SETS AND TRUE ARITHMETIC

By Post's theorem, Σ_n^0 definable sets are exactly those which are m -reducible to the n th jump $\emptyset^{(n)}$, and moreover, any set computable in $\emptyset^{(n)}$ is Δ_{n+1}^0 . Thus the arithmetically definable sets of natural numbers are exactly those which are computable in some degree $0^{(n)}$. Consider the ascending chain of degrees

$$0 <_T 0^{(1)} <_T 0^{(2)} < \dots$$

We can get an upper bound for this chain by forming the effective join

$$\emptyset^{(\omega)} = \bigoplus_{n < \omega} \emptyset^{(n)} = \{\langle i, n \rangle \mid i \in \emptyset^{(n)}\}$$

The associated degree $0^{(\omega)}$ is called True Arithmetic because it contains $\text{Th}(\mathbb{N})$ (and $\text{Th}(L_\omega)$).

This degree is an upper bound on the arithmetic sets in the sense that any arithmetic set is computable in it. Our goal will be to see that there are upper bounds strictly below $0^{(\omega)}$, and to find out just how much lower they can be.

2. UPPER BOUNDS BELOW TRUE ARITHMETIC

Note that an upper bound X on the arithmetic sets must satisfy $\emptyset^{(n)} \leq_T X$ for every n , as these are arithmetic sets, and conversely, any set computing these computes all arithmetic sets by Post's theorem. Thus an upper bound on arithmetic sets can uniformly compute all sets at a fixed level of the arithmetic hierarchy. The way in which such a set can be weaker than true arithmetic is that it can fail to have a uniform way to compute $\emptyset^{(n)}$ as n varies. This idea guides our first construction.

Theorem 1 (Kleene-Post 1954). *There is a set $X \subset \omega$ such that $X <_T 0^{(\omega)}$, but $0^{(n)} \leq_T X$ for all n .*

Proof. The idea is to view the set we are constructing as a grid. We will put the sets $\emptyset^{(n)}$ into rows in the grid, but we will do so in such a way that there is no uniformly computable way to find the row where we put $\emptyset^{(n)}$ given n . In order to disguise these meaningful rows, we will put some additional information in other rows.

This will be formalized as a kind of forcing construction, where we build an approximation of X step by step so as to meet dense conditions. We describe our approximations with strings. The alphabet Σ has a symbol f for every finite set $f \in [\omega]^{<\omega}$, and a formal symbol x_n representing $\emptyset^{(n)}$. Given a finite or infinite string σ in this alphabet, we define $X(\sigma)$ to be set of pairs

$$\{\langle i, k \rangle \mid i < |\sigma| \text{ and } k \in \sigma(i)\}$$

where $k \in x_n$ means $k \in \emptyset^{(n)}$. The string tells us how to fill in rows to build $X(\sigma)$. In the case that σ is finite and n is largest such that x_n occurs in σ , $X(\sigma) \leq_T \emptyset^{(n)}$, and moreover, this reduction is uniform in σ . The X we want will be of the form $X(g)$ where $g \in \Sigma^\omega$ is infinite. We construct g so as to satisfy the following requirements:

$$\begin{aligned} R_n &: X(g) \geq_T \emptyset^{(n)} \\ S_e &: \Phi_e^{X(g)} \neq \emptyset^{(\omega)} \end{aligned}$$

It suffices to show that given any finite string σ and any requirement, we can extend σ to a larger finite string τ so that for any g extending τ , the requirement is satisfied. Then we can simply line up the requirements

and satisfy them one by one to build g . The requirements R_n are easy to satisfy, simply append x_n to the end of σ .

We now consider extending σ so as to satisfy S_e , so we are comparing $\Phi_e^{X(g)}$ and $\emptyset^{(\omega)}$. I claim that there is an $i \in \omega$ such that one of the following two things is true:

- (1) For any $g \succeq \sigma$, $\Phi_e^{X(g)}(i) \uparrow$.
- (2) For some extension $\tau = \sigma \frown f_1 \dots f_r$, $\Phi_e^{X(\tau)}(i) \downarrow \neq \emptyset^{(\omega)}(i)$, and the computation only queries rows below $|\tau|$.

Suppose this is not true. Then for any i , there is some extension of σ making $\Phi_e^{X(g)}(i) \downarrow$. This uses only a finite amount of $X(g)$, so we can take it to be of the form $\sigma \frown f_1 \dots f_r$. Because (2) does not hold, given any such extension, it halts with the correct value for $\emptyset^{(\omega)}$. This allows us to compute $\emptyset^{(\omega)}$ from $X(\sigma)$ by searching extensions of σ , which is a contradiction as $X(\sigma) \leq_T \emptyset^{(n)}$ for some n .

Fix an i such that either (1) or (2) holds. If (1) holds, S_e is already satisfied. If (2) holds, we can extend σ to $\sigma \frown f_1 \dots f_r$ to satisfy S_e . We conclude that we can make finite extensions of strings to satisfy the requirements, and so we can construct an infinite g satisfying all of them. For this g , we have ensured that $X(g) \geq_T \emptyset^{(n)}$ for all n , and $X(g) \not\geq_T \emptyset^{(\omega)}$. The last thing is to observe that $\emptyset^{(\omega)}$ can effectively carry out the steps in the construction of g , and so we can choose g such that $X(g) <_T \emptyset^{(\omega)}$. $\square$

3. FORCING THE JUMP

We will now improve the above theorem, to put some space between X and $\emptyset^{(\omega)}$.

Theorem 2. *There is an X such that for all n , $0^{(n)} \leq_T X$, but $X^{(1)} \leq_T 0^{(\omega)}$, so we have*

$$0 <_T 0^{(1)} <_T 0^{(2)} <_T \dots <_T X <_T X^{(1)} \leq_T 0^{(\omega)}$$

The idea of this is a generally applicable idea in forcing constructions, *forcing the jump*. As we construct g , we will not only determine the bits of $X(g)$, but also the bits of $X(g)^{(1)}$. Because $0^{(\omega)}$ can follow the construction, it will compute $X(g)^{(1)}$.

Proof. I could set up a whole new construction to do this forcing the jump argument, but it is easier to point out that this already happens in the original argument. We can identify the e th bit of $X(g)^{(1)}$ with the truth value of the statement $\Phi_e^{X(g)}(e) \downarrow$. There is a computable function h such that the machine $\Phi_{h(e)}$, given any oracle Y and input i , ignores its input and computes $\Phi_e^Y(e)$. In the step of the construction of g where we handle the requirement $S_{h(e)}$, we determine whether or not $\Phi_{h(e)}^{X(g)}$ halts on some input, and so we determine whether or not $\Phi_e^{X(g)}(e) \downarrow$. $\square$

In this argument, we are using the machinery of forcing to determine the truth of Σ_1^0 properties of X using its finite approximations. Formally, we could replace each S_e with step where we decide the e th bit of $X^{(1)}$. These bits determine the truth of all Σ_1^0 properties of X .

4. IMPLICIT DEFINABILITY OF THE JUMP

At this point, it is natural to ask the following question.

Question. What is the largest $\alpha \leq \omega$ such that there is an X satisfying $X \geq_T 0^{(n)}$ for all n , and $X^{(\alpha)} \leq_T 0^{(\omega)}$?

$$0 <_T 0^{(1)} <_T \dots <_T X <_T X^{(1)} <_T \dots <_T X^{(\alpha)} \leq_T 0^{(\omega)}$$

In other words, how much additional power do we get by being able to compute the $\emptyset^{(n)}$ uniformly, as opposed to merely being able to compute each one individually. To start to answer this question, we have the following upper bound.

Theorem 3 (Enderton-Putnam 1970). *If $X \geq_T 0^{(n)}$ for all n , then $X^{(2)} \geq_T 0^{(\omega)}$.*

The idea is that if X contains the information of every $\emptyset^{(n)}$, then because the Turing jump itself is an implicitly arithmetically definable operation, we can reconstruct the sequence $\emptyset, \emptyset^{(1)}, \emptyset^{(2)}, \dots$ in a uniform way given a few more jumps of power.

Proof. A partial computable function being total is a Π_2^0 property, so $X^{(2)}$ knows for which e the machine Φ_e^X is total. Thus in $X^{(2)}$, we have a uniform enumeration of all X computable sets: define

$$Y_e = \begin{cases} \{i \mid \Phi_e^X(i) = 1\} & \Phi_e^X \text{ is total} \\ \emptyset & \text{otherwise} \end{cases}$$

This list contains all of the $\emptyset^{(n)}$. To compute $\emptyset^{(\omega)}$, we just need to compute a function f such that $\emptyset^{(n)} = Y_{f(n)}$ for all n . We can do this because $X^{(2)}$ also knows which X -computable sets are the jumps of other X -computable sets. Indeed, consider the relation

$$\{(i, j) \mid Y_j = Y_i^{(1)}\}$$

I claim that this is computable in $X^{(2)}$. First, one checks whether Φ_i^X and Φ_j^X are total, and acts accordingly if they are not. Assuming both are total,

$$Y_j = Y_i^{(1)} \iff (\forall n) \left[\Phi_n^{\Phi_i^X}(n) \downarrow \longrightarrow \Phi_j(n) = 1 \text{ and } \Phi_n^{\Phi_i^X}(n) \uparrow \longrightarrow \Phi_j(n) = 0 \right]$$

One can check that this is Π_2^0 relative to X . Now $X^{(2)}$ can compute indices for the sequence $\emptyset, \emptyset^{(1)}, \emptyset^{(2)}, \dots$ as follows. Start with an e such that $\Phi_e^X = \emptyset$. Suppose we have compute indices $i_0, \dots, i_{n-1}$ such that $Y_{i_k} = \emptyset^{(k)}$. Then search for a j such that $Y_j = Y_{i_{n-1}}^{(1)}$. This lets us recursively compute the desired function f . $\square$

5. AN OPTIMAL UPPER BOUND BELOW TRUE ARITHMETIC

At this point we know that the answer to our question is either 1 or 2. The following theorem of Sacks tells us that the answer is 2.

Theorem 4 (Sacks 1971). *There is an X such that $X \geq_T 0^{(n)}$ for all n , and $X^{(2)} \equiv_T 0^{(\omega)}$.*

From the point of view of the previous proof, the problem is to find a way to determine the truth of Σ_2^0 properties of X while encoding all $0^{(n)}$ into X . Previously we determined the truth of Σ_1^0 properties of X . The solution to this problem is a more elaborate forcing construction, using Sacks forcing. In this notion of forcing, the approximating forcing conditions are certain subtrees of $2^{<\omega}$.

Definition. Let $T \subset 2^{<\omega}$ be a subtree of the full binary tree.

- (1) T is *perfect* if every node has a branch node extending it. In particular, there are no dead ends.
- (2) T is *pointed* if every path π in T computes T itself.

Definition. A *Sacks forcing condition* is an arithmetically definable pointed perfect subtree $P \subset 2^{<\omega}$. Given Sacks forcing conditions P, Q , we say that Q extends P , $P \leq Q$, if Q is a subtree of P . For $\pi \in 2^\omega$, we say $P \leq \pi$ if π is a path in P .

In this notion of forcing, we view the path π as the target object approximated by the tree P . There is a weakest Sacks forcing condition, the full tree $2^{<\omega}$. For the purposes of computation relative to $0^{(\omega)}$, Sacks forcing conditions can be represented by pairs (n, e) where $n < \omega$ and e is an index such that $\Phi_e^{0^{(n)}}$ computes the tree. One needs to be a bit careful here: $0^{(\omega)}$ cannot determine which pairs represent Sacks forcing conditions because the property of being pointed is Π_1^1 , but relative to $0^{(\omega)}$, we can manipulate codes which happen to represent Sacks forcing conditions. We will see in our argument that this is enough, all the conditions we deal with will be constructed so as to be pointed.

A useful observation about Sacks forcing conditions is that any perfect tree has a leftmost path which is computable in the tree, so if $P \leq Q$, then Q computes P . In the other direction, if Q is a perfect subtree of a Sacks forcing condition P and $Q \leq_T P$, then Q is also pointed, and so is a Sacks forcing condition extending P .

The idea of the proof will be to describe a $0^{(\omega)}$ -effective construction of a sequence of Sacks forcing conditions

$$P_0 \leq P_1 \leq P_2 \leq \dots$$

converging to a unique path π . At each stage, we either ensure that π computes $0^{(n)}$ for some n , or we decide the truth of some Σ_2^0 property of π . If we do this for all n and all Σ_2^0 properties of π , then $0^{(\omega)}$ computes

$\pi^{(2)}$, and $\pi^{(2)}$ computes $0^{(\omega)}$ by the previous theorem, so π is the object we want. To carry this out, we need two lemmas.

Lemma 1. *Given P a Sacks forcing condition and n , there is a $Q \geq P$ such that any path through Q computes $0^{(n)}$.*

Lemma 2. *Given P a Sacks forcing condition and $\varphi(X)$ a Σ_2^0 predicate on 2^ω , there is a $Q \geq P$ such that one of the following is true:*

- (1) *every path through Q satisfies φ*
- (2) *every path through Q satisfies $\neg\varphi$*

We will see in the proofs of these lemmas that $0^{(\omega)}$ can effectively find such a Q given P , and effectively determine which case we are in. This is enough to complete the proof of the theorem using the strategy described previously.

Sketch of Proof of Lemma 1. This is where pointedness is useful. Any path in Q a subtree of P is a path in P , and so computes P . Thus it suffices to ensure that for any path π in Q , $\pi \oplus P$ computes $Q \oplus 0^{(n)}$. We can do this by making π go in certain directions at branch points of P to encode bits of $Q \oplus 0^{(n)}$. We can index the branch points of P by $2^{<\omega}$. We leave alone branch points corresponding to strings of even length, to ensure that Q is perfect. At branch points corresponding to strings of odd length, we cut off either the left or right branch to encode a bit of $Q \oplus 0^{(n)}$. Of course, this is circular, because we are constructing Q at the same time that we are encoding its bits, but we have plenty of space to do this. The resulting Q is computable in $P \oplus 0^{(n)}$, and can be found effectively in $0^{(\omega)}$ given n and a code for P . $\square$

The proof of Lemma 2 is based on introducing a second kind of forcing. We will refer to this as the “local” forcing construction, in contrast to the “global” forcing construction using Sacks conditions.

Definition. Given P a Sacks forcing condition, a P -local forcing condition is a string $\sigma \in P$. These strings are ordered by extension.

Here we think of strings as approximations of paths in P in the usual sense. We will want to keep careful track of how these conditions determine arithmetical properties of the path they approximate. To do this, we introduce the *forcing language*. This has atomic formulas $R_e^X(y_1, \dots, y_r)$, for Δ_0^0 -predicates $R_e^X(y_1, \dots, y_r)$. We build sentences by plugging in numbers for the y_i or applying quantifiers $\forall y_i, \exists y_i$ or negation. In particular, any Σ_2^0 fact about X can be expressed by a sentence in this language of the form

$$(\exists x)(\forall y)R_e^X(x, y)$$

for some choice of e . We define the local forcing relation $\sigma \Vdash_P \varphi$, where $\sigma \in P$ and φ is a sentence in the forcing language, as follows:

- (1) $\sigma \Vdash_P R_e^X(n_1, \dots, n_r)$ if $R_e^\sigma(n_1, \dots, n_r)$ evaluates to true, querying only bits of σ .
- (2) $\sigma \Vdash_P (\exists x)\varphi(x)$ if for some number n , $\sigma \Vdash_P \varphi(n)$.
- (3) $\sigma \Vdash_P (\forall x)\varphi(x)$ if and only if for any $\tau \succeq \sigma$ in P and any n , there is some $\varrho \succeq \tau$ in P such that $\varrho \Vdash_P \varphi(n)$.
- (4) $\sigma \Vdash_P \neg\varphi$ if for every $\tau \succeq \sigma$ in P , $\tau \nVdash_P \varphi$.

We think of this relation as expressing the idea that if we know that σ is an initial segment of a path π , then π will satisfy φ so long as π is “sufficiently generic,” how this bears out exactly for Σ_2^0 properties will be seen below. It follows from the definition that for any σ, φ , either $\sigma \Vdash_P \neg\varphi$ or for some $\tau \succeq \sigma$, $\tau \Vdash_P \varphi$. The last property of the local forcing relation $\Vdash_P$ we will need is that it is monotone. If $\sigma \Vdash_P \varphi$, and $\tau \succeq \sigma$ in P , then $\tau \Vdash_P \varphi$.

Proof of Lemma 2. It will turn out in the proof that $Q \leq_T P$. This takes care of the requirements that Q is pointed and arithmetically definable. We just need to ensure that Q is perfect and decide a truth value of a given Σ_2^0 sentence.

Fix a Σ_2^0 sentence

$$\varphi = (\exists x)(\forall y)R_e^X(x, y)$$

As remarked previously, at least one of the following is true

- (1) There is some $\sigma \in P$ such that $\sigma \Vdash_P \varphi$.
- (2) There is some $\sigma \in P$ such that $\sigma \Vdash_P \neg\varphi$.

Moreover, determining which one is true is uniformly computable relative to $0^{(\omega)}$ given φ and (a code for) P . How we construct Q will depend on which case we are in.

Case 1. Suppose $\sigma \in P$ satisfies $\sigma \Vdash_P \varphi$. We will construct a perfect subtree $Q \subset P$, every path of which passes through σ such that $Q \leq_T P$ and every path π through Q satisfies φ . Unraveling the forcing relation for φ , we can fix some n such that $\sigma \Vdash_P (\forall y) R_e^X(n, y)$. Therefore, for every $\tau \succeq \sigma$ in P and every m , there is a $\varrho \succeq \tau$ in P such that $R_e^e(n, m)$ evaluates to true. Using μ -recursion, we have a P -computable function

$$(\tau, m) \mapsto \varrho$$

witnessing this, which we can take to output branch nodes of P . To build $Q \leq_T P$, it suffices to enumerate the branch nodes of Q in order of their heights. We do this recursively, starting with an arbitrary branch node $\sigma_\lambda \succeq \sigma$. Given a branch node τ of height m , we effectively branch nodes $\tau_0 \succeq \tau \frown 0$, $\tau_1 \succeq \tau \frown 1$ in P such that $\tau_i \Vdash_P R_e^X(n, m)$ for each i . We declare these to be branch nodes in Q of height $m+1$. Iterating this enumerates a set of branch nodes of P , defining a P -computable perfect subtree Q . Now we check that for any path π in Q satisfies φ , witnessed by n . This is because π passes through branch nodes of Q of all heights, and so $R_e^e(n, m)$ is true for all m . Therefore, π satisfies φ .

Case 2. Suppose $\sigma \in P$ satisfies $\sigma \Vdash_P \neg\varphi$. We will construct a perfect subtree $Q \subset P$ as before, such that every path π through Q satisfies $\neg\varphi$. This time, unraveling the local forcing relation tells us that for every n and every $\tau \succeq \sigma$ in P , there is an m and a $\varrho \succeq \tau$ in P such that $R_e^e(n, m)$ evaluates to false. Again, we have a P -computable function

$$(\tau, n) \mapsto (\varrho, m)$$

witnessing this, which we can assume outputs branch nodes of P in the ϱ slot. We then enumerate branch nodes of Q as in the previous case. Now because any path π in Q passes through branch nodes of all heights, we have that for any n , there is an m such that $R_e^e(n, m)$ is false, and so π satisfies $\neg\varphi$.

Examining the construction, we see that $0^{(\omega)}$ can effectively determine whether or not we will make φ or $\neg\varphi$ true of all paths, can effectively find Q , given φ and a code for P . $\square$

With these lemmas, the proof of Sacks' theorem is complete. I close with a quote from Sacks, in his paper.

“Cohen taught that truth can be approximated more easily and more completely in the intermediate stages of a construction than any recursion theorist would have believed possible, and Spector revealed that infinite approximation by means of perfect closed conditions is radically different than finite approximation.”